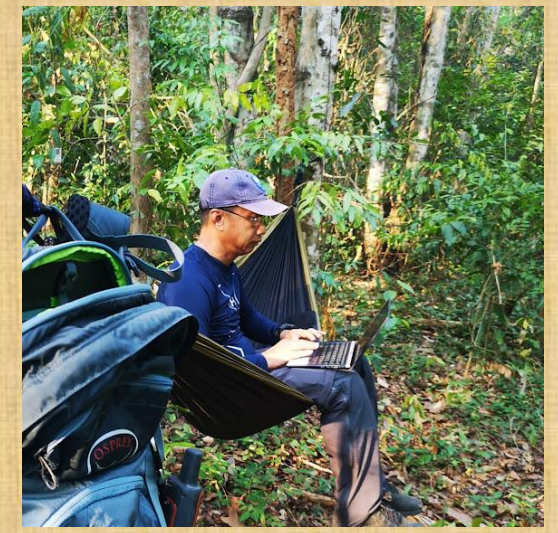
ITCS214 : DATA STRUCTURES & ALGORITHM ANALYSIS



Lecture 7 · Sorting Algorithms





Asst. Prof. Dr. Ananta Srisuphab
ananta.sri@mahidol.ac.th
2026/1

Mahidol University
Faculty of Information and Communication Technology

What you will be able to do

- TRACE — run each of the four algorithms on paper, showing before → operation → after for every step.
- EXPLAIN — argue why each algorithm is correct, using its loop invariant.
- IMPLEMENT — turn the pseudocode into working code that uses O(1) auxiliary space.
- ANALYZE — count comparisons, shifts and swaps, and derive best / average / worst-case Θ bounds.
- CHOOSE — pick the right algorithm for a scenario (nearly sorted data, expensive writes, …) and defend the choice.

Why sorting, and the words we'll use

Why sorting?

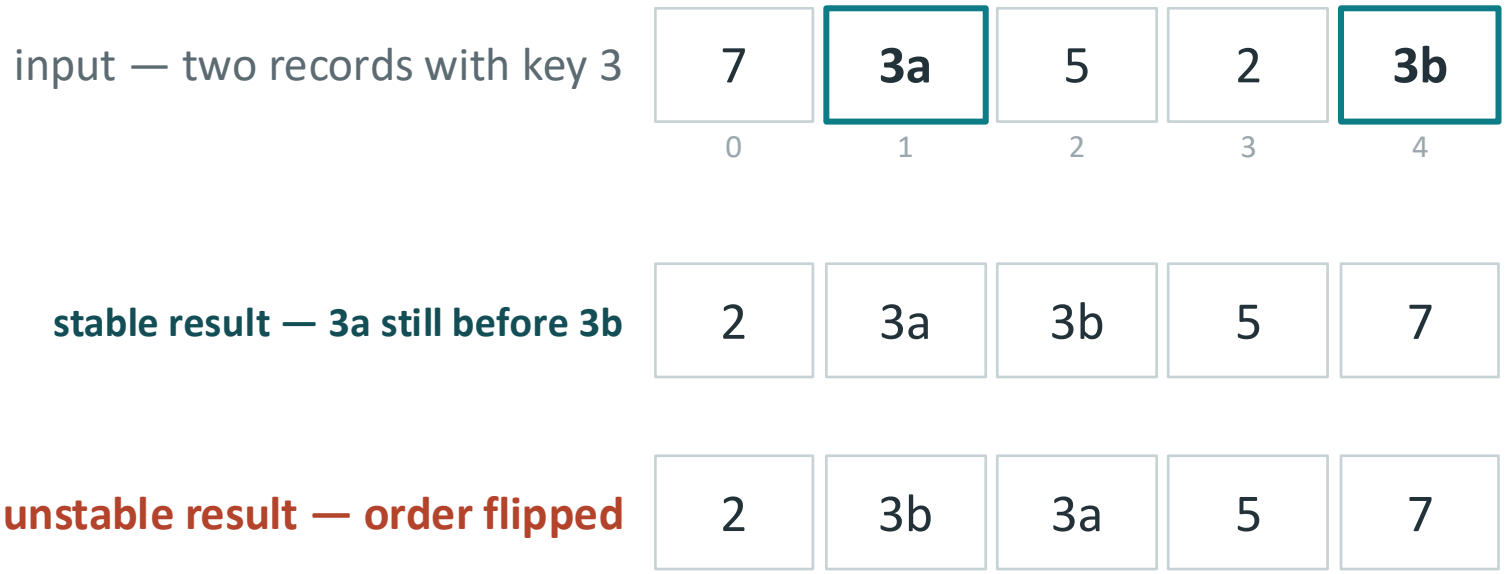
- Binary search needs sorted data — sorting is the pre-processing step of half of computing.
- Grouping, ranking, duplicate detection all start with a sort.
- Real libraries still run insertion sort on small ranges inside their fast sorts.
- On some hardware, writes are expensive — the sort you choose decides how many you pay.

Conventions for today

- We sort records by a key. Example: student records sorted by ID.
- Ascending order: $a[0] \le a[1] \le \ldots \le a[n-1]$.
- Arrays are 0-based; n is the length.
- A “pass” means something different in each algorithm — we define it when each algorithm is introduced.

Stability — equal keys keep their order

A sort is stable if records with equal keys end up in the same relative order they started in.



Labels a, b only identify the records — comparisons see the key 3 on both.

Today: insertion & bubble are stable (they act only on strict >). Selection & Shell are not guaranteed stable.

Space, adaptiveness, and what we count

In place

- Sorts inside the array itself.
- Auxiliary space O(1): a few variables (i, j, key/temp).
- **All four algorithms today are in place.**

Adaptive

- Runs faster when the input is already (nearly) in order.
- **Insertion: yes. Bubble: only with its flag.**
- Selection: no. Shell: partly.

What we count

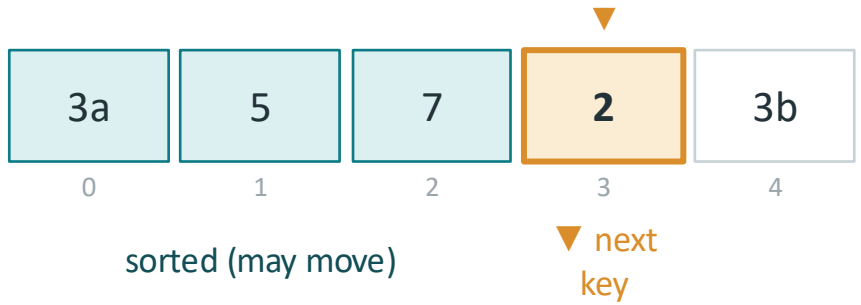
- Comparison: is a[j] > key?
- Shift: copy one slot right — 1 array write.
- Swap: exchange two slots — 3 assignments, 2 array writes.

Each operation costs O(1) time, so total running time grows in proportion to these counts — counting operations is how we will explain every Θ bound today.

Like sorting cards in your hand

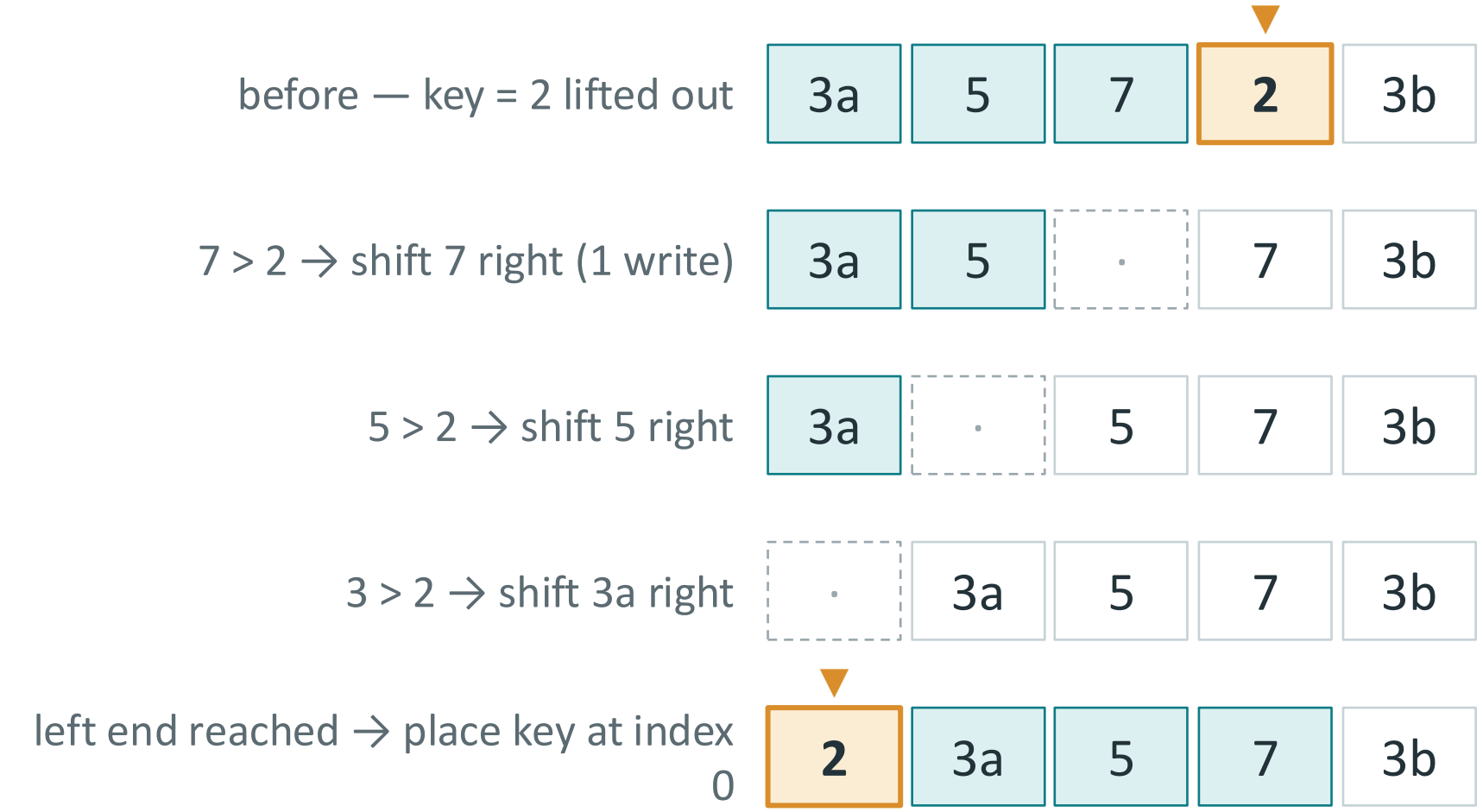
- Pick up the next card (the key); slide it left past every bigger card until it fits.
- **Pass i = inserting the key a[i] into the sorted prefix, for i = 1 … n−1.**
- Property maintained: after pass i, the prefix a[0..i] is sorted…
- **…but NOT final — a later, smaller key can still push these elements to the right.**

state after pass 2 — about to start pass 3:



sorted (may move) key unsorted

One insertion in slow motion — pass 3, key = 2



cost of this pass: 3 comparisons, 3 shifts, 1 placement — the key waits in a temp variable the whole time

The remaining passes, summarized

after pass 1 (key 3a)	3a	7	5	2	3b
after pass 2 (key 5)	3a	5	7	2	3b
after pass 3 (key 2)	2	3a	5	7	3b
after pass 4 (key 3b)	2	**3a**	**3b**	5	7

Stability, caught in the act: in pass 4, key 3b scans left and meets 3a. The test is a[j] > key, and 3 > 3 is false — so 3b stops and lands after 3a. Moving only on strict > is exactly what makes insertion sort stable.

Pseudocode and why it works

```
INSERTION-SORT(a, n)              // 0-based
  for i = 1 to n-1                // pass i
    key = a[i]
    j = i - 1
    while j >= 0 and a[j] > key
      a[j+1] = a[j]               // shift
      j = j - 1
    a[j+1] = key
```

strict > ⇒ stable · variables key, i, j only ⇒ O(1) auxiliary space

Loop invariant

- **Before pass i: a[0..i−1] is sorted.**
- The while loop shifts exactly the prefix elements greater than key one slot right — opening the hole at the key's correct place.
- Placing key there leaves a[0..i] sorted → the invariant holds for i+1.
- After the last pass (i = n−1): a[0..n−1] is sorted. ■

Cost — count the comparisons and shifts

Case	Comparisons	Shifts	Time
Best — already sorted	$n - 1$ (one per pass)	0	$\Theta(n)$
Worst — reverse sorted	$\approx n^2/2$	$\approx n^2/2$	$\Theta(n^2)$
Average — random order	$\approx n^2/4$	$\approx n^2/4$	$\Theta(n^2)$

Adaptive: time is $\Theta(n + I)$, I = number of inversions

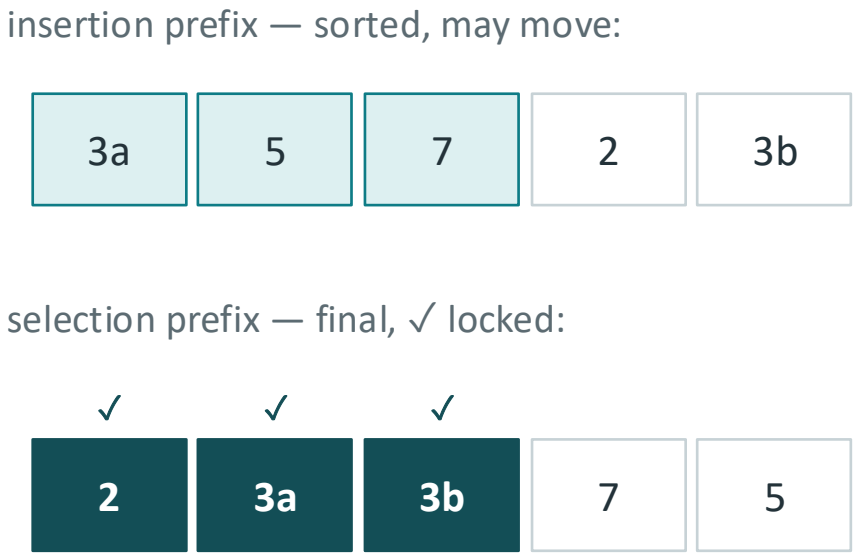
$O(1)$ auxiliary space

Stable (strict >)

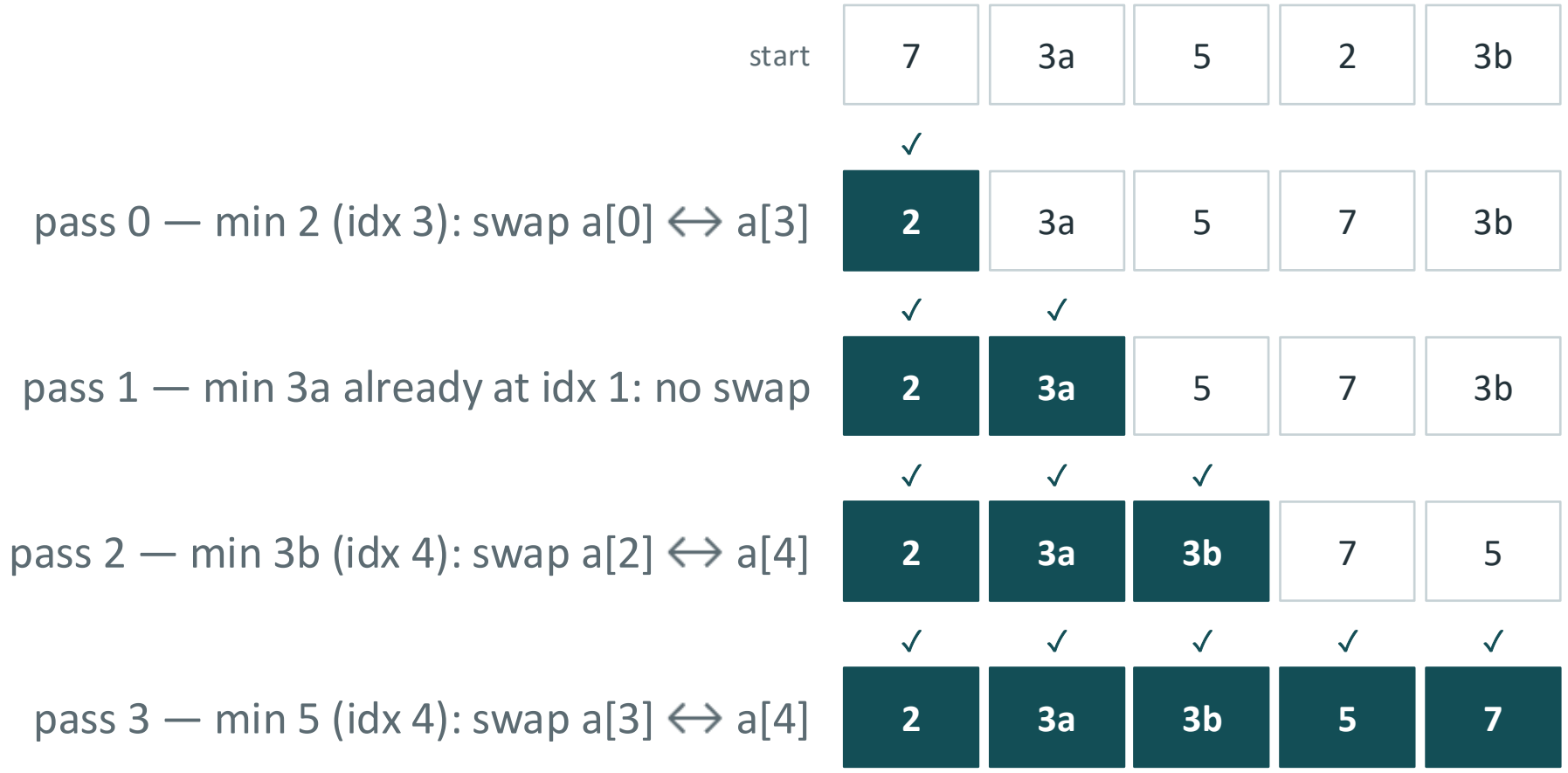
An inversion is a pair out of order ($i < j$ but $a[i] > a[j]$). Every shift fixes exactly one inversion — so nearly-sorted data (few inversions) sorts in nearly linear time. The average case assumes all input orders are equally likely.

Pick the minimum, lock it in

- Scan the unsorted part for its minimum; swap it into the next free slot.
- **Pass i = filling index i with the minimum of a[i..n−1], for i = 0 … n−2.**
- Property maintained: after pass i, the prefix a[0..i] holds the i+1 smallest keys, in sorted order, in their FINAL positions.
- **Those cells never move again — unlike insertion's prefix.**



All passes on the shared array



The equal 3s stayed in order here — that was luck, not a guarantee. Proof on the next slides.

Pseudocode and why it works

```
SELECTION-SORT(a, n)              // 0-based
  for i = 0 to n-2                // pass i
    m = i                        // min index
    for j = i+1 to n-1
      if a[j] < a[m]
        m = j
    if m != i              // no self-swap
      swap a[i], a[m]
```

Loop invariant

- **Before pass i: a[0..i−1] holds the i smallest keys, sorted, final.**
- The inner loop ends with m = index of the minimum of a[i..n−1].
- The swap puts that (i+1)-th smallest key at index i → invariant holds for i+1.
- When i = n−1 remains, the last slot must hold the maximum. ■

swap when m ≠ i only ⇒ at most n−1 swaps · O(1) auxiliary space

The same work, every single time

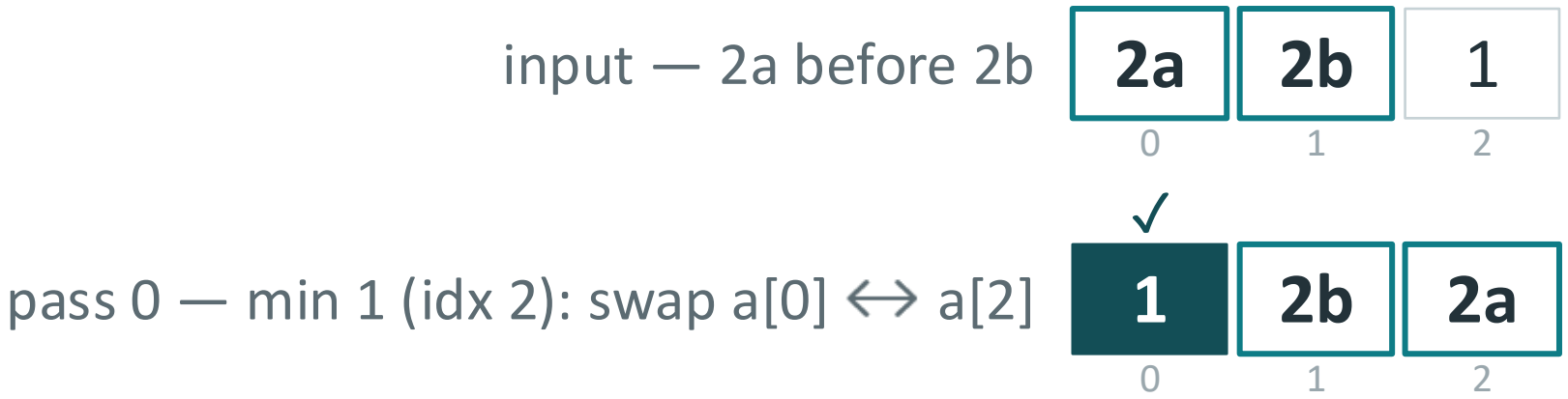
$$n(n-1)/2$$

key comparisons — on every input.
For n = 10: exactly 45, sorted or shuffled.

- **The inner loop's length depends only on i, never on the data → best = average = worst = $\Theta(n^2)$ comparisons.**
- Not adaptive: a sorted input gets no discount.
- **But: at most n−1 swaps (self-swaps skipped) — minimal data movement. Our 10-element demo used just 7.**
- Auxiliary space O(1); unstable in general (next slide).

Rule of thumb: selection sort's comparisons dominate its time; its (few) swaps dominate its writes.

Why selection sort is not stable



The long-distance swap threw 2a over 2b. The records with equal key 2 finish in reversed order — one counterexample is enough: standard swap-based selection sort is not stable. Any algorithm that moves elements far in a single jump runs this risk. Remember that when we meet Shell sort.

Neighbors swap; the largest floats right

- Compare adjacent cells; swap whenever left > right.
- **Pass = one left-to-right sweep over the not-yet-final region.**
- During a pass, the largest remaining key is picked up and carried to the region's right end.
- **Property maintained: after k completed passes, the last k slots hold the k largest keys — final ✓, a sorted suffix growing from the right.**

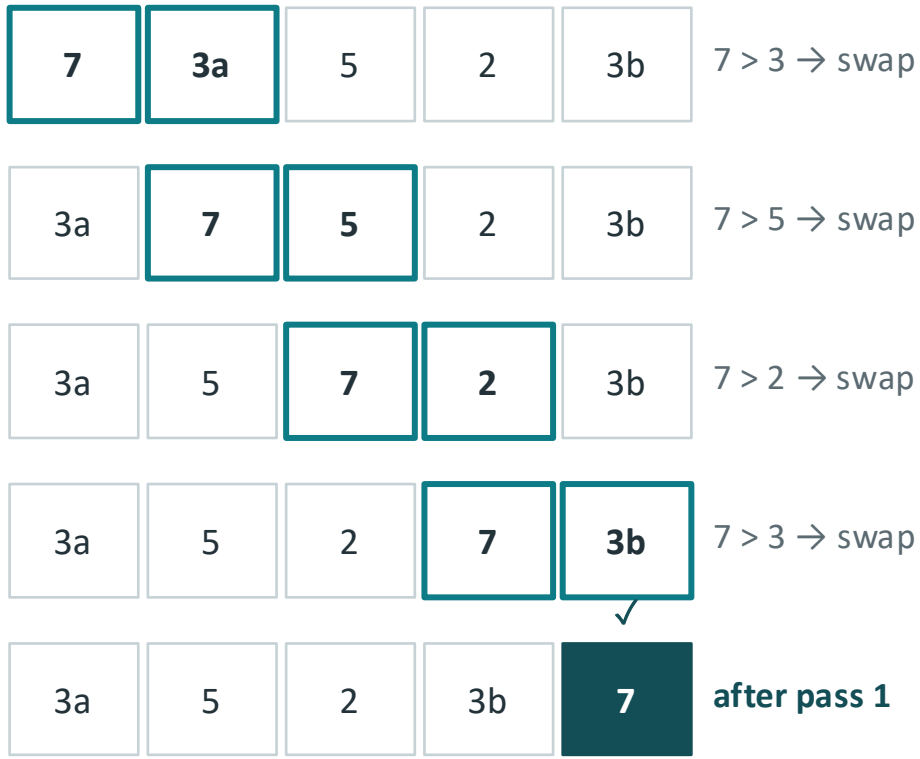
after 2 passes on the shared array:



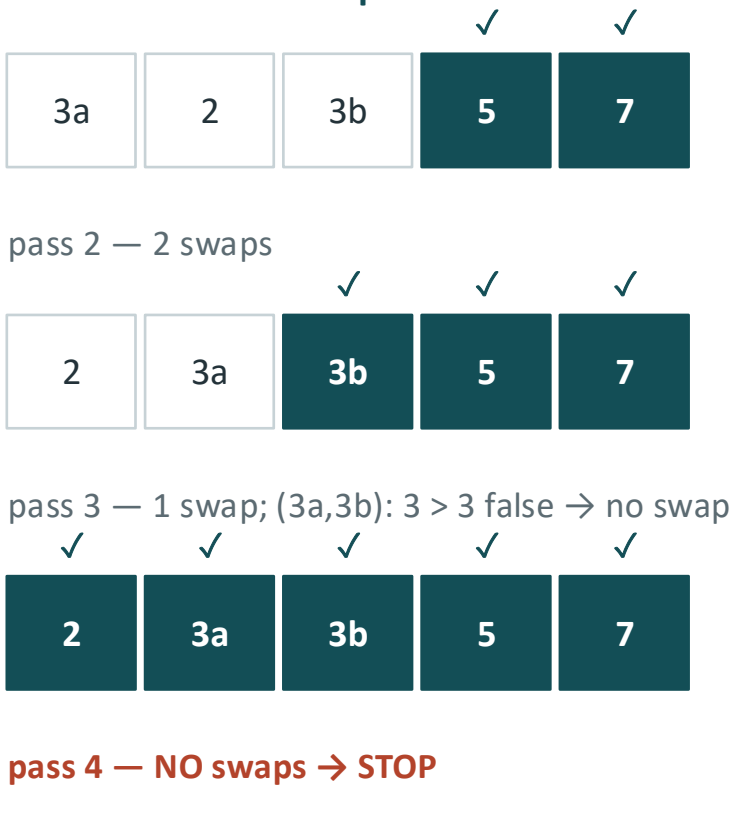
suffix = final, grows right → left

Pass 1 in detail, passes 2–4 summarized

pass 1 — every comparison:



state after each later pass:



pass 4 — NO swaps → STOP

each pass sweeps a shorter region — the ✓suffix is never re-visited

Pseudocode — the flag and the shrinking boundary

```
BUBBLE-SORT(a, n)                  // 0-based
  for i = 0 to n-2
    swapped = false
    for j = 0 to n-2-i             // last i
                                   // slots final
      if a[j] > a[j+1]             // strict >
        swap a[j], a[j+1]
        swapped = true
    if not swapped
      break                        // early stop
```

Why it works

- Invariant: after pass i+1, the last i+1 slots hold the largest keys, final — so j stops at n−2−i.
- **Early stop is sound: a pass with no swaps means every adjacent pair is ordered ⇒ the whole array is sorted.**
- Strict > ⇒ equal neighbors never swap ⇒ stable.

Cost — the flag earns the best case

Case	Comparisons	Swaps	Time
Best — sorted, WITH flag	$n - 1$ (single pass)	0	**$\Theta(n)$**
Best — sorted, without flag	$\approx n^2/2$	0	**$\Theta(n^2)$**
Worst — reverse sorted	$\approx n^2/2$	$\approx n^2/2$	**$\Theta(n^2)$**
Average — random order	$\Theta(n^2)$	$\approx n^2/4$	**$\Theta(n^2)$**

Stable (strict >)

O(1) auxiliary space

Adaptive only through the flag

Versus selection: the same $\Theta(n^2)$ comparisons in the worst case, but up to $\approx n^2/2$ swaps — an order of magnitude more array writes. Bubble's honest niches: teaching, and a cheap “is it already sorted?” check (one flagged pass).

Insertion sort's weakness, and Shell's fix

- Insertion moves a key ONE slot per shift — an element far from home costs many writes.
- **Shell's idea (1959): first sort interleaved subsequences with a big stride g, so one shift jumps g slots.**
- Shrink g and repeat; finish with g = 1 — plain insertion sort on an almost-sorted array.
- g-sorted: every subsequence of indices {r, r+g, r+2g, …} is in order.
- **Invariant: after the g-stage, the array is g-sorted. The g = 1 stage makes it fully sorted.**

Our gap sequence (n = 10)

integer halving:

g = 5 → 2 → 1

- 10/2 = 5; 5/2 = 2; 2/2 = 1 (integer division).
- Other gap sequences exist — and change the running time (later).

Gap 5 — five interleaved subsequences



A = {0, 5}: (29, 8) — 29 > 8 → shift 29, place 8

B = {1, 6}: (10, 21) — in order

C = {2, 7}: (14, 40) — in order

D = {3, 8}: (37, 33) — 37 > 33 → shift 37, place 33

E = {4, 9}: (13, 19) — in order



Each subsequence here has only TWO elements, so one comparison settles it. That is a coincidence of n = 10, g = 5 — not the general rule. Next slide: the general rule.

The engine: insertion sort with stride g

```
SHELL-SORT(a, n)                    // 0-based
  g = n / 2                         // 5, 2, 1
  while g >= 1
    for i = g to n-1                // gapped
      key = a[i]                    // insertion
      j = i
      while j >= g and a[j-g] > key
        a[j] = a[j-g]               // shift by g
        j = j - g                   // may repeat!
      a[j] = key
    g = g / 2                       // integer div
```

Read it as insertion sort

- **Replace every “1” in insertion sort with “g” — that's the whole change.**
- The inner while can shift SEVERAL elements of the same subsequence, one g-jump each — exactly like slide 8, with longer steps.
- **A single comparison-and-swap per index is NOT sufficient in general.**
- g = 1 makes the last stage literal insertion sort — correctness follows.

Gap 2 — inserting 19 takes three shifts

subsequences at g = 2: even indices {0,2,4,6,8} and odd indices {1,3,5,7,9} — 19 belongs to the odd one



THREE shifts inside one subsequence — a single compare-and-swap could never produce this. Last row = end of the gap-2 stage.

Gap 1 finishes cheaply — and the honest cost picture



only 14 and 21 are out of place — one single-slot shift each → sorted:



Complexity — always relative to the gap sequence and implementation

- This halving version — best case (already sorted): each of the ~log n stages does ≈ n comparisons, 0 shifts → Θ(n log n). Worst case: $\Theta(n^2)$. Average: no simple formula — we will not quote one.
- **Unstable (long jumps) · O(1) auxiliary space · better gap sequences give better bounds · NOT guaranteed to beat insertion / selection / bubble — measure on your data.**

The four algorithms, side by side

Algorithm	Best	Average	Worst	Space	Stable	Adaptive	Reach for it when…
Insertion	Θ(n)	Θ(n²)	Θ(n²)	O(1)	Yes	Yes — Θ(n+I)	small n · nearly sorted · data arrives one at a time
Selection	Θ(n²)	Θ(n²)	Θ(n²)	O(1)	No	No	writes are expensive — at most n−1 swaps
Bubble (flag)	Θ(n)*	Θ(n²)	Θ(n²)	O(1)	Yes	Via flag	teaching · cheap "already sorted?" check
Shell (halving)	Θ(n log n)	no simple formula	Θ(n²)	O(1)	No	Partly	medium arrays where insertion is too slow

** only with the swapped flag. Stability as implemented on these slides (strict > comparisons). Shell's bounds are for the integer-halving gap sequence; other sequences differ.*

No single winner — every column is a trade-off. The next slide makes you choose.